

Fiche TP — Semaine 3

Les formulaires du MiniForum

Développement web, classe passerelle

Durée 4h

Rendu avant S4

Objectif du TP

Mettre en place les **quatre formulaires** du MiniForum — création de sujet, réponse à un sujet, inscription et connexion — puis vérifier que l'ensemble du site reste utilisable **au clavier** et par un lecteur d'écran.

Avant de commencer

Ce TP suppose que les notions du polycopié *S03_polycop* (champs, attributs, name contre id, validation native, <label>, <fieldset>, accessibilité) ont été vues en cours. Garde-le à portée de main.

Tu travailles dans le dépôt miniforum, avec les deux pages `index.html` et `sujet.html` produites en semaine 2.

Comment lire cette fiche

Comme la semaine dernière, cette fiche ne donne pas le code à recopier. Elle décrit le **rendu attendu** sous forme de maquette et précise ce qu'il faut produire. À toi d'écrire le HTML correspondant.

Les balises annotées sur les maquettes (<fieldset>, <label>...) sont des indications, pas du texte à afficher. Les champs y sont dessinés vides : c'est bien leur **structure** qui est demandée, pas leur contenu.

Étape		Ce que tu produis	Temps
1–2	Reprise et création de sujet	le formulaire principal du forum	1h
3–4	Réponse et inscription	deux formulaires de plus	1h15
5–6	Connexion et navigation	les quatre formulaires reliés	40 min
7	Passage au clavier et aux DevTools	un site réellement utilisable	45 min
8	Validation et envoi	un dépôt propre et validé	20 min

1 Reprendre le projet

1. Ouvrir le dossier `miniforum` dans VS Code et récupérer les éventuelles mises à jour :

```
git pull
```

2. Ouvrir `index.html` avec *Live Server* (clic droit → *Open with Live Server*) et garder la page visible à côté de l'éditeur pendant tout le TP.
3. Relire rapidement le HTML de la semaine 2 : les formulaires vont s'ajouter **dans** la structure existante, sans la casser.

Toujours pas de CSS

Les formulaires vont être franchement laids : champs collés les uns aux autres, largeurs inégales, boutons gris. C'est normal et c'est voulu.

Un formulaire sans style force à juger sa **structure** — et c'est exactement ce qui est évalué cette semaine. La mise en forme arrive en semaine 4.

2 Le formulaire de création de sujet

C'est le formulaire principal du forum : celui qui permet de lancer une nouvelle discussion. Il vient s'ajouter dans le `<main>` de `index.html`, **après** la section des sujets.

The diagram illustrates the structure of the 'Nouveau sujet' form in `index.html`. It shows the following elements and their corresponding HTML tags:

- Nouveau sujet** (Title, `<h2>`)
- Votre message** (Fieldset, `<fieldset> + <legend>`)
 - Pseudo** (Text input, `<input type="text"> + <label>`, placeholder 'ex : alice')
 - Titre du sujet** (Text input, `<input type="text">`, required)
 - Message** (Text area, `<textarea>`)
- Options de publication** (Fieldset, `<fieldset>`)
 - Catégorie** (Dropdown, `<select> -- trois <option>`)
 - Visibilité** (Radio buttons, `<input type="radio">`, values 'Public' and 'Réservé aux membres', 'Public' is selected)
- Publier le sujet** (Submit button, `<button type="submit">`)

À produire dans `index.html`

Une `<section>` contenant un titre `<h2>` « Nouveau sujet » et un `<form>` avec `action="/topics"` et `method="post"`, composé de :

- un premier `<fieldset>` « Votre message » avec trois champs :
 - **Pseudo** — input de type `text`, `name="pseudo"`, obligatoire, avec un placeholder d'exemple ;
 - **Titre du sujet** — input de type `text`, `name="titre"`, obligatoire, limité à 80 caractères ;
 - **Message** — `textarea` de `name="message"`, obligatoire, 6 lignes, limité à 500 caractères ;
- un second `<fieldset>` « Options de publication » avec :
 - une liste déroulante **Catégorie** (`name="categorie"`) proposant *Général*, *Aide* et *Projets* ;
 - deux boutons radio **Visibilité** (`name="visibilite"`), valeurs `public` et `prive`, le premier coché par défaut ;
- un bouton d'envoi `<button type="submit">`.

Chaque champ a un `<label>` relié par `for / id`, et un `name` distinct.

Les name de ce formulaire sont définitifs

pseudo, titre, message, categorie, visibilite : ces cinq noms sont exactement les clés que le serveur Express lira en semaine 15, sous la forme `req.body.titre`, `req.body.message`...

Choisis-les en minuscules, sans accent ni espace, et ne les change plus.

Le premier test à faire

Sauvegarde, puis clique sur « Publier le sujet » avec les champs **vides**. Le navigateur doit refuser l'envoi, placer le curseur dans le premier champ obligatoire et afficher un message.

Si la page se recharge sans rien dire, c'est qu'il manque les `required`.

Point d'étape

```
git add index.html
git commit -m "Formulaire de creation de sujet"
```

3 Le formulaire de réponse

Dans `sujet.html`, sous la liste des messages, on ajoute de quoi répondre à la discussion. Ce formulaire est plus court — deux champs suffisent.

... sujet.html

Répondre à ce sujet `<h3>`

Votre réponse

Pseudo `name="pseudo"`

Message `<textarea name="message">`

Envoyer la réponse `<button type="submit">`

À produire dans `sujet.html`

Un `<form>` placé après la section des réponses, avec :

- un titre `<h3>` « Répondre à ce sujet » ;
- un `<fieldset>` « Votre réponse » contenant un champ **Pseudo** (`name="pseudo"`) et un **Message** (`textarea, name="message"`), tous deux obligatoires ;
- un bouton d'envoi.

Attention aux `id` en double

Le formulaire de `sujet.html` utilise les mêmes **noms de champs** que celui de `index.html` — c'est normal, ce sont les mêmes données. Mais les deux formulaires sont dans **deux pages différentes** : aucun conflit d'`id` n'est possible.

En revanche, si tu ajoutes plus tard un second formulaire **dans la même page**, les `id` devront être distincts (`pseudo-reponse`, `message-reponse`...) alors que les `name` pourront rester identiques. C'est un bon test de compréhension de la différence entre les deux attributs.

4 La page d'inscription

C'est le formulaire le plus riche du projet : il rassemble presque tous les types d'input vus en cours. Crée un nouveau fichier `inscription.html` à la racine, en reprenant le squelette et le `<header>` des pages existantes.

À produire dans `inscription.html`

La même charpente que les autres pages (`<header>`, `<main>`, `<footer>`), avec dans le `<main>` un `<h2>` « Créer un compte » et un `<form action="/inscription" method="post">` contenant :

- un `<fieldset>` « Identifiants » :
 - Pseudo** — texte, `name="pseudo"`, obligatoire, avec un `pattern` de 3 à 15 lettres ou chiffres et un `title` qui explique la règle ;
 - Adresse e-mail** — `type="email"`, `name="email"`, obligatoire ;
 - Mot de passe** — `type="password"`, `name="motdepasse"`, obligatoire, `minlength="8"` ;
 - Date de naissance** — `type="date"`, `name="naissance"` ;
- un `<fieldset>` « Préférences » avec deux cases à cocher : la lettre d'information (facultative, `name="newsletter"`) et l'acceptation des conditions (**obligatoire**, `name="cgu"`) ;
- un bouton « Créer mon compte » et un lien vers `connexion.html`.

Le `title` n'est pas facultatif

Sans `title`, un `pattern` qui refuse la saisie affiche « Veuillez respecter le format demandé » — et l'utilisateur reste bloqué sans savoir quoi corriger.

Teste ton champ `pseudo` avec une saisie invalide (ab, ou alice!) : le message affiché doit être le **tien**.

Aucun mot de passe n'est protégé aujourd'hui

`type="password"` masque les caractères à l'écran. C'est tout. La valeur n'est ni chiffrée, ni protégée, et elle transiterait en clair vers le serveur.

Le hachage des mots de passe est traité en **semaine 18**. D'ici là, n'utilise jamais un vrai mot de passe personnel pour tester tes pages.

5 La page de connexion

Le plus simple des quatre. Crée `connexion.html` sur le même modèle.

À produire dans `connexion.html`

Un `<form action="/connexion" method="post">` avec un `<fieldset>` « Vos identifiants » contenant un champ **identifiant** (texte, obligatoire), un champ **mot de passe** (obligatoire) et une case à cocher « Rester connecté ». Un bouton d'envoi, et un lien vers `inscription.html`.

Pourquoi `method="post"` ici, sans discussion

Avec `method="get"`, le mot de passe apparaîtrait **dans la barre d'adresse** — donc dans l'historique du navigateur, dans les journaux du serveur et dans le presse-papier de quiconque copie l'URL.

C'est le cas d'école qui justifie `post`. Retiens-le : il tombera au contrôle écrit.

6 Relier les quatre pages

Quatre formulaires existent, mais rien ne permet encore de les atteindre.

1. Dans le `<nav>` des **quatre** pages, ajouter deux entrées : « Inscription » vers `inscription.html` et « Connexion » vers `connexion.html`.
2. Vérifier que le `<nav>` est identique partout : même ordre, mêmes intitulés.
3. Tester tous les parcours dans le navigateur : accueil → sujet → inscription → connexion → accueil.

Le copier-coller a une limite

Quatre pages partagent maintenant le même en-tête. Si tu modifies le menu, il faut le modifier **quatre fois** — et l'oubli d'une page est la source d'erreur classique.

C'est une vraie limite du HTML statique, et non un défaut de ta méthode. Elle disparaîtra en semaine 16, quand le serveur générera les pages à partir d'un modèle commun.

7 Passer le site à l'accessibilité

Les formulaires fonctionnent. Reste à vérifier qu'ils sont utilisables par tout le monde. Cette partie compte autant que les précédentes.

Le test du clic sur l'étiquette

1. Sur chaque page, cliquer sur **chaque** texte d'étiquette — pas sur le champ.
2. Le curseur doit se placer dans le champ correspondant, ou la case se cocher.
3. Si rien ne se passe : le `for` du `<label>` ne correspond pas à l'id du champ. Corrige.

Une seconde par champ

C'est le test le plus rentable de la semaine : il détecte en quelques secondes la totalité des liaisons `label / input` cassées, que le validateur W3C ne signale pas toujours.

Le test des trente secondes au clavier

1. Cliquer dans la barre d'adresse du navigateur, puis **lâcher la souris**.
2. Parcourir toute la page avec Tab, et remplir un formulaire entier sans souris : Tab pour avancer, Espace pour cocher, flèches pour les boutons radio, Entrée pour envoyer.
3. Vérifier les trois points ci-dessous.

Ce qui doit être vrai	Sinon
Tu vois toujours où tu te trouves	Un cadre doit entourer l'élément actif : ne le supprime jamais en CSS
L'ordre suit la lecture naturelle	L'ordre de tabulation suit l'ordre du code : réordonne le HTML
Tu atteins tous les champs et boutons	Un élément inatteignable au clavier est inutilisable par une partie des visiteurs

Le vérificateur d'accessibilité des DevTools

1. Ouvrir les outils de développement (F12), onglet *Accessibilité* (ou *Accessibility*).
2. Sélectionner un champ dans la page et lire l'**arbre d'accessibilité** : le champ doit apparaître avec un nom — celui de son `<label>`. Un champ dont le nom est vide est un champ inutilisable sans écran.
3. Lancer un audit *Lighthouse* en cochant uniquement *Accessibility*, sur les quatre pages.
4. Corriger tous les problèmes signalés comme bloquants, puis relancer.

Un score de 100 ne prouve rien

Une machine sait vérifier qu'un alt **existe**, pas qu'il **décrit** l'image. Elle sait qu'un `<label>` est relié, pas qu'il est compréhensible.

Le score est un plancher, pas un objectif : le test du clavier reste le seul qui compte vraiment.

8 Valider et sauvegarder**Valider les quatre pages**

1. Aller sur `validator.w3.org`, onglet *Validate by Direct Input*.
2. Coller le contenu de chaque page, l'une après l'autre, et corriger en commençant par **la première erreur de la liste**.

Message fréquent cette semaine	Correction
The for attribute of the label element must refer to a form control	Le <code>for</code> pointe vers un id qui n'existe pas : corriger l'un ou l'autre
Duplicate ID "pseudo"	Deux champs portent le même id dans une page : en renommer un
Element legend not allowed as child of element form	Le <code><legend></code> doit être dans un <code><fieldset></code> , en première position
Attribute maxlength not allowed on element select	Cet attribut n'existe pas sur <code><select></code> : le supprimer
Bad value for attribute pattern	L'expression régulière est mal écrite : vérifier les crochets et les accolades

Envoyer sur GitHub

```
git add .
git commit -m "Formulaires d'inscription et de connexion, corrections d'accessibilite"
git push
```

Vérifier que c'est bien parti

Sur `github.com`, ton dépôt doit contenir **quatre** fichiers HTML, et l'onglet *Commits* doit montrer plusieurs commits pour cette séance — un par étape, pas un seul en fin de TP.

9 Pour aller plus loin (optionnel)

Si tu as terminé en avance :

- Ajouter un champ **Rechercher un sujet** dans le `<header>` de `index.html`, avec `type="search"` et `method="get"` — le seul cas du projet où `get` est le bon choix. Explique pourquoi en commentaire.
- Remplacer la liste déroulante des catégories par un `<datalist>` : l'utilisateur peut alors choisir *ou* saisir librement.
- Ajouter un `<input type="hidden">` dans le formulaire de réponse pour transporter l'identifiant du sujet, et réfléchir à ce que « caché » signifie exactement — le champ est-il invisible pour l'utilisateur, ou introuvable ?

- Regrouper les `<option>` de la catégorie par thème avec `<optgroup>`.
- Installer un lecteur d'écran (NVDA sous Windows, VoiceOver déjà présent sous macOS) et parcourir un formulaire les yeux fermés. C'est l'exercice qui change le plus la façon d'écrire du HTML.

10 Points de contrôle

Auto-évaluation

- ☐ Les quatre pages existent : `index.html`, `sujet.html`, `inscription.html`, `connexion.html`
- ☐ Chaque formulaire est dans une balise `<form>` avec `action` et `method`
- ☐ `method="post"` partout, sauf éventuellement pour une recherche
- ☐ **Chaque** champ a un `name` en minuscules, sans accent ni espace
- ☐ **Chaque** champ a un `<label>` relié par `for / id`, testé au clic
- ☐ Les champs obligatoires portent `required`, et l'envoi à vide est bien refusé
- ☐ Le champ pseudo de l'inscription a un `pattern` **et** un `title` explicite
- ☐ Les types sont adaptés : `email`, `password`, `date`
- ☐ Les champs liés sont groupés dans un `<fieldset>` avec un `<legend>`
- ☐ Les deux boutons radio de la visibilité partagent le même `name` et des `value` distinctes
- ☐ Chaque `<button>` a un attribut `type` explicite
- ☐ Aucun `id` n'est en double dans une même page
- ☐ Le `<nav>` des quatre pages est identique et tous les liens fonctionnent
- ☐ Chaque page se parcourt et se remplit entièrement **au clavier**, avec un focus visible
- ☐ Les quatre pages passent `validator.w3.org` **sans erreur**
- ☐ L'audit Lighthouse ne signale plus de problème d'accessibilité bloquant
- ☐ Le travail est poussé sur GitHub, en **plusieurs** commits au message clair

À rendre avant la semaine 4

Le dépôt GitHub `miniforum` à jour, avec les quatre pages validées et navigables au clavier.

La semaine 4 ouvre le **CSS**. Les trois semaines de structure que tu viens de terminer vont enfin recevoir leur mise en forme — **sans une seule ligne de HTML à retoucher**. C'est précisément ce que permet un HTML bien écrit.

Bloqué ?

Si une étape résiste plus de dix minutes, demande. Le polycopié *S03_polycop* contient la réponse à la quasi-totalité des questions de ce TP — commence par y chercher, puis appelle.